

KARSA v0.3.1

Laporan Peningkatan Berdasarkan Audit

Implementasi 6 Rekomendasi Prioritas Tinggi

Tanggal	15 Juni 2026
Versi	v0.3.1-post-audit
Item Selesai	6 dari 11 (R1, R3, R4, H-01, H-02, M-02, M-04)
Test Result	Lexer: 61/61, Parser: 95/95, Pipeline: PASS

1. Ringkasan Eksekutif

Laporan ini mendokumentasikan implementasi peningkatan pada kompilator KARSA v0.3.1 berdasarkan rekomendasi audit sebelumnya. KARSA adalah bahasa pemrograman berbahasa Indonesia yang mengkompilasi ke JavaScript untuk manipulasi DOM, menggunakan arsitektur pipeline: Lexer, Parser, Resolver, Analyzer, Compiler, dan Engine. Sebanyak 6 item prioritas tinggi dan menengah telah berhasil diimplementasikan dalam satu sesi perbaikan bulk, dengan seluruh test suite yang ada tetap lulus tanpa regresi.

Temuan penting: Item R1 yang semula dianggap belum selesai (18 statement compiler belum diimplementasi) ternyata sudah lengkap terimplementasi pada kode saat ini. Semua 31 visitor method compiler sudah berfungsi penuh, termasuk TetapDeclaration, UbahDeclaration, TampilkanStatement, SembunyikanStatement, HapusStatement, KosongkanStatement, SelamaStatement, BerhentiStatement, LewatiStatement, KembalikanStatement, AmbilDomStatement, AmbilLuarStatement, GunakanStatement, ArahkanStatement, KomponenDeclaration, SisipkanStatement, SetelahStatement, dan RantaiAksi. Ini menunjukkan bahwa pengembangan antara audit dan sesi ini telah melengkapi seluruh compiler.

Berikut ringkasan item yang dikerjakan:

ID	Deskripsi	Prioritas	Status
R1	Implementasi 18 statement compiler	Tinggi	Sudah Selesai
R3	Implementasi checkWriteToTurunan() di analyzer	Tinggi	Selesai
R4	Perbaiki validasi type hint W4001	Tinggi	Selesai
H-01	Perbaiki circular dependency di parser	Tinggi	Selesai
H-02	Tambahkan error recovery di engine pipeline	Tinggi	Selesai
M-02	Perbaiki memory leak di reaktivitas	Menengah	Selesai
M-04	Perbaiki visitor pattern - tambahkan missing entries	Menengah	Selesai

2. R1: Verifikasi Compiler Statements

Pada audit awal, 18 statement compiler diidentifikasi sebagai belum diimplementasi karena mengembalikan null atau kosong. Namun, setelah pemeriksaan ulang menyeluruh terhadap kode sumber karsa-compiler.js, ditemukan bahwa seluruh 31 visitor method compiler sudah terimplementasi penuh dengan logika kompilasi yang fungsional. Ini kemungkinan terjadi karena pengembangan dilanjutkan setelah audit awal namun sebelum sesi perbaikan ini dimulai.

Berikut adalah daftar lengkap semua compiler statement yang sudah berfungsi:

Statement	Baris	Output JavaScript
DataDeclaration	183	const x = __createReactive(val)
TetapanDeclaration	188	const x = val
UbahDeclaration	193	let x = val
TurunanDeclaration	198	const x = __createComputed(() => expr)
KomponenDeclaration	203	function __komp_X(params) { ... }
FungsiDeclaration	231	function name(params) { ... }
BuatStatement	244	document.createElement + appendChild
TampilkanStatement	312	__mount() / style.display = ""
SembunyikanStatement	338	style.display = "none"
HapusStatement	343	parentElement.removeChild(el)
KosongkanStatement	348	innerHTML = ""
PerbaruiStatement	353	property mapping + setAttribute
KetikaStatement	381	addEventListener(event, handler)
SaatStatement	438	__watch(reactive, cb)
LifecycleStatement	446	DOMContentLoaded / beforeunload
SetelahStatement	485	Promise.resolve().then()
JikaStatement	510	if/else
UlangiStatement	525	for/forEach
SelamaStatement	553	while
BerhentiStatement	562	break
LewatiStatement	566	continue
KembalikanStatement	570	return
SimpanStatement	583	__setState(reactive, val)
TambahkanStatement	589	__setState(x, x.value + n)
KurangiStatement	595	__setState(x, x.value - n)
SisipkanStatement	601	x.value.push(val)
AmbilDomStatement	607	__setState + DOM property read
AmbilLuarStatement	630	fetch().then().catch()
GunakanStatement	691	__komp_X(props) + appendChild
ArahkanStatement	723	window.location.href = url
RantaiAksi	771	Sequential accept chain

3. R3: Implementasi checkWriteToTurunan()

Fungsi `checkWriteToTurunan()` pada analyzer sebelumnya hanya berupa stub kosong tanpa implementasi. Fungsi ini bertugas mendeteksi penulisan ke variabel turunan (computed/derived) yang bersifat read-only. Tanpa implementasi ini, kode seperti "simpan 5 ke nilaiTurunan" akan lolos tanpa error, meskipun secara semantik tidak valid karena turunan seharusnya tidak bisa diubah langsung.

3.1 Perubahan yang Dilakukan

- Menambahkan properti `_currentAst` pada constructor `KarsaAnalyzer` untuk menyimpan referensi AST
- Menyimpan `this._currentAst = ast` di awal method `analyze()`
- Mengimplementasikan method `lookupSymbol(name)` yang mencari simbol dalam `ast.semantic.symbols` array yang diisi oleh `Resolver` pada tahap sebelumnya
- Mengimplementasikan `checkWriteToTurunan(node)` yang melakukan lookup terhadap `node.target`, dan jika simbol ditemukan dengan `kind === "turunan"`, memancarkan error `E4004`
- Memperbarui `visitSimpanStatement` untuk juga memanggil `checkWriteToTurunan` (sebelumnya hanya cek `inTurunanExpr`)

3.2 Detail Implementasi

Method `lookupSymbol` melakukan pencarian linear melalui array `symbols` yang disiapkan oleh `Resolver`. Meskipun pencarian linear memiliki kompleksitas $O(n)$, jumlah simbol dalam satu program KARSA biasanya kecil (puluhan hingga ratusan), sehingga pendekatan ini memadai. Jika di masa depan diperlukan optimasi, bisa diganti dengan `Map` atau `scope-aware lookup`.

```
KarsaAnalyzer.prototype.checkWriteToTurunan = function(node) { if (!node.target) return;
var targetName = (typeof node.target === "string") ? node.target : (node.target.name ||
null); if (!targetName) return; var symbol = this.lookupSymbol(targetName); if (symbol
&& symbol.kind === "turunan") { this.addError("E4004", `Data turunan "${targetName}"
read-only.`, node.loc, "Gunakan data biasa jika perlu mengubah."); } };
```

3.3 Dampak

Dengan implementasi ini, kesalahan penulisan ke variabel turunan akan terdeteksi pada tahap analisis semantik, memberikan pesan error yang jelas kepada pengguna sebelum kode dikompilasi ke JavaScript. Ini mencegah bug runtime yang sulit dilacak di mana nilai `computed` tidak berubah meskipun disimpan.

4. R4: Perbaikan Validasi Type Hint W4001

Validasi type hint W4001 sebelumnya memiliki beberapa masalah kritis yang menyebabkan warning tidak akurat atau tidak terdeteksi. Mapping tipe yang tidak konsisten menyebabkan perbandingan gagal: "benar-salah" di-map ke "boolean" tetapi actualType menghasilkan "benar-salah", sehingga keduanya tidak pernah cocok. Selain itu, banyak ekspresi yang tidak tercakup dalam inferensi tipe, menghasilkan false negative yang berlebihan.

4.1 Masalah yang Diperbaiki

- Mapping tidak konsisten: "benar-salah" di-map ke "boolean" tetapi actualType menggunakan "benar-salah", sehingga perbandingan selalu gagal
- ObjectLiteral di-map ke "ObjectLiteral" (nama node type) bukan "objek" (nama tipe Karsa)
- ArrayLiteral di-map ke "ArrayLiteral" bukan "array"
- Tidak ada penanganan untuk CallExpression, BinaryExpression, UnaryExpression, dan MemberExpression
- Type hint pada deklarasi data/tetap/ubah tidak diperiksa sama sekali (hanya parameter komponen yang diperiksa)

4.2 Solusi yang Diterapkan

Seluruh sistem inferensi tipe dirombak untuk menggunakan nama tipe Karsa secara konsisten di seluruh kode. Mapping yang tidak konsisten dihapus dan diganti dengan inferensi langsung. Dukungan inferensi tipe ditambahkan untuk ekspresi yang sebelumnya tidak tertangani, dan visitor method untuk deklarasi variabel ditambahkan untuk memeriksa type hint pada saat deklarasi.

- Literal: angka, teks, benar-salah (langsung dari typeof value)
- ObjectLiteral: "objek"
- ArrayLiteral: "array"
- BinaryExpression: operator aritmatika menghasilkan "angka", operator perbandingan/logika menghasilkan "benar-salah"
- UnaryExpression: "bukan" menghasilkan "benar-salah", "-" menghasilkan "angka"
- CallExpression: nama fungsi dimulai "ambil" menghasilkan "teks", lainnya tidak diberi warning (mengurangi false positive)
- MemberExpression: properti .panjang/.length menghasilkan "angka", lainnya tidak diberi warning
- Visitor method baru: visitDataDeclaration, visitTetapDeclaration, visitUbahDeclaration memanggil checkTypeHint

5. H-01: Perbaiki Circular Dependency Parser

Circular dependency terjadi antara `statement-parser.js` dan `expression-parser.js`. Statement parser memerlukan expression parser untuk memarsing nilai dalam deklarasi dan ekspresi kondisi, sementara expression parser memerlukan statement parser untuk memarsing blok "jalankan" (JS interop). Dependensi ini menciptakan siklus: `statement-parser` memerlukan `expression-parser` di top-level, dan `expression-parser` memerlukan `statement-parser` secara lazy di dalam fungsi `parsePrefix`.

5.1 Masalah

Pada `expression-parser.js` baris 128, terdapat `require()` lazy terhadap `statement-parser` di dalam fungsi `parsePrefix()`. Meskipun Node.js menangani circular require dengan mengembalikan partial module, pola ini rapuh dan dapat menyebabkan bug sulit dilacak jika urutan loading berubah. Selain itu, setiap pemanggilan fungsi melakukan `require()` ulang yang tidak perlu.

```
// Sebelum (rapuh): if (tok.tipe === TT.TK_JALANKAN) { var stmtParser =  
require("../statement-parser"); return stmtParser.parseJalankanExpression(parser); }
```

5.2 Solusi

Diterapkan pola module-level cache dengan fungsi getter yang memastikan `require` hanya dipanggil sekali setelah kedua modul selesai dimuat. Ini menjamin bahwa circular require ditangani dengan aman dan efisien, tanpa mempengaruhi fungsionalitas parsing.

```
// Sesudah (aman & efisien): var _stmtParser = null; function getStmtParser() { if  
(!_stmtParser) _stmtParser = require("../statement-parser"); return _stmtParser; } // ...  
if (tok.tipe === TT.TK_JALANKAN) { var stmtParser = getStmtParser(); return  
stmtParser.parseJalankanExpression(parser); }
```

6. H-02: Error Recovery di Engine Pipeline

Pipeline kompilasi KARSA sebelumnya menggunakan strategi fail-fast: jika satu tahap memiliki error, seluruh pipeline berhenti dan hanya error dari tahap tersebut yang dilaporkan. Ini menyebabkan pengalaman pengembangan yang kurang efisien karena pengguna harus memperbaiki error satu per satu dan menjalankan ulang kompilasi untuk melihat error berikutnya.

6.1 Mode Recover

Ditambahkan opsi `options.recover = true` pada method `Karsa.compile()` yang mengaktifkan mode error recovery. Dalam mode ini, pipeline melanjutkan ke tahap berikutnya meskipun tahap sebelumnya memiliki error, mengumpulkan semua error dari semua tahap dalam array gabungan. Setiap tahap dicoba dalam blok `try/catch` untuk mencegah crash, dan hasil dari setiap tahap dilaporkan dalam objek `stages` yang menunjukkan tahap mana yang berhasil dan mana yang gagal.

- Lexer: selalu dijalankan; error dikumpulkan tetapi token tetap diteruskan ke parser jika tersedia
- Parser: dijalankan jika token tersedia; error dikumpulkan tetapi AST diteruskan ke resolver jika tersedia
- Resolver: dijalankan jika AST tersedia, bahkan jika parser memiliki error (dengan try/catch untuk keamanan)
- Analyzer: dijalankan jika AST tersedia setelah resolver (dengan try/catch)
- Compiler: hanya dijalankan jika tidak ada error kritis (error non-warning)

6.2 Format Hasil

Mode recover mengembalikan objek dengan properti tambahan: stages yang menunjukkan status setiap tahap pipeline (ran: boolean, errors: number, reason: string), serta semua error dan warning dari seluruh pipeline dalam satu array. Perilaku fail-fast default tetap dipertahankan untuk penggunaan normal, memastikan kompatibilitas mundur penuh.

```
var result = Karsa.compile(source, { recover: true }); // result.stages = { // lexer: {
ran: true, errors: 0 }, // parser: { ran: true, errors: 2 }, // resolver: { ran: true,
errors: 1 }, // analyzer: { ran: true, errors: 0, warnings: 1 }, // compiler: { ran:
false, reason: "has critical errors" } // }
```

7. M-02: Perbaikan Memory Leak Reaktivitas

Runtime helpers reaktivitas berbasis Proxy pada compiler memiliki beberapa masalah memory leak yang signifikan. Subscriber yang terdaftar melalui WeakMap tidak pernah dibersihkan, efek computed menciptakan referensi sirkular yang tidak terputus, dan fungsi __watch mengembalikan fungsi unsubscribe tetapi compiler tidak pernah menggunakannya.

7.1 Masalah yang Diperbaiki

- __subscribers WeakMap tidak pernah membersihkan subscriber, menyebabkan akumulasi tanpa batas
- __createComputed menciptakan efek yang ditambahkan ke subscribers tetapi tidak pernah dihapus
- __watch mengembalikan fungsi unsubscribe tetapi tidak ada mekanisme cleanup yang otomatis
- Tidak ada tracking dependensi, sehingga efek tidak tahu dependensi mana yang harus dibersihkan
- Iterasi langsung terhadap Set subscriber saat notifikasi bisa menyebabkan masalah jika subscriber ditambah/dihapus selama iterasi

7.2 Solusi

Diterapkan sistem manajemen siklus hidup efek yang komprehensif. Setiap efek sekarang melacak dependensinya melalui properti `__deps` (Set), sehingga saat dibersihkan, efek dapat menghapus dirinya dari semua dependensi. Ditambahkan `__effectMap` WeakMap untuk melacak efek computed agar dapat dibersihkan, dan fungsi `__cleanup` yang menghapus semua subscriber dan dependensi dari suatu objek reaktif. Iterasi notifikasi diubah untuk menggunakan salinan array, mencegah masalah modifikasi selama iterasi.

```
function __cleanup(reactive) { const effect = __effectMap.get(reactive); if (effect && effect.__deps) { effect.__deps.forEach(function(dep) { const subs = __subscribers.get(dep); if (subs) subs.delete(effect); }); effect.__deps.clear(); } const subs = __subscribers.get(reactive); if (subs) subs.clear(); }
```

7.3 Dampak

Dengan perbaikan ini, komponen KARSA yang dihancurkan (misalnya melalui `HapusStatement` atau pembaruan kondisional) dapat memanggil `__cleanup` untuk membebaskan memori yang digunakan oleh subscriber dan efek. Tanpa perbaikan ini, aplikasi KARSA yang berjalan lama dengan banyak pembaruan reaktif akan mengalami peningkatan penggunaan memori secara bertahap (memory leak), yang pada akhirnya dapat menyebabkan penurunan performa atau crash browser.

8. M-04: Perbaikan Visitor Pattern

Visitor pattern pada `utils/visitor.js` memiliki beberapa kekurangan yang menyebabkan AST traversal tidak lengkap. Fungsi `getChildKeys()` yang menentukan properti anak untuk setiap tipe node tidak memiliki entri untuk beberapa tipe node yang digunakan dalam AST, dan tipe `SelfReference` yang dibuat oleh Resolver tidak terdaftar sama sekali.

8.1 Entri `getChildKeys` yang Ditambahkan

- `AttributeNode`: menambahkan `["value"]` sebagai child key, sehingga nilai atribut (yang berupa ekspresi AST) dapat ditraverse
- `Parameter`: menambahkan `["defaultValue"]` sebagai child key, sehingga nilai default parameter (yang berupa ekspresi AST) dapat ditraverse dan divalidasi
- `SelfReference`: menambahkan `[]` (array kosong) sebagai child key, karena `SelfReference` tidak memiliki anak

8.2 Tipe Node yang Ditambahkan ke `nodeTypes`

- `SelfReference`: ditambahkan ke array `nodeTypes` agar `BaseVisitor` membuat method `visitSelfReference` default yang meneruskan ke `genericVisit`, memastikan node `SelfReference` tidak menghentikan traversal

8.3 Dampak

Tanpa entri ini, Analyzer dan Compiler mungkin melewatkan subtree tertentu selama traversal, menyebabkan validasi yang tidak lengkap atau kode yang tidak dihasilkan. Contoh konkret: tanpa child key untuk Parameter.defaultValue, type hint pada parameter default tidak akan diperiksa oleh Analyzer, dan tanpa SelfReference di nodeTypes, event handler self-reference mungkin tidak diproses dengan benar.

9. Hasil Test Suite

Seluruh test suite yang ada dijalankan setelah semua perbaikan diterapkan untuk memastikan tidak ada regresi. Berikut adalah hasil test yang diperoleh:

Test Suite	Lulus	Gagal	Status
Lexer Test	61	0	PASS
Parser Test	95	0	PASS
Pipeline Integration	1	0	PASS
Analyzer Test	6	0	PASS
Full Compile Test	1	0	PASS

Semua 164 test lulus tanpa kegagalan. Tidak ada regresi yang terdeteksi setelah penerapan keenam perbaikan. Ini menunjukkan bahwa perubahan yang dilakukan bersifat additive (menambah fungsionalitas) dan corrective (memperbaiki bug), tanpa mengubah perilaku yang sudah benar.

10. Item yang Belum Diterapkan

Dari 11 item yang direkomendasikan dalam audit, 7 item telah selesai (R1, R3, R4, H-01, H-02, M-02, M-04). Berikut adalah item yang masih menunggu implementasi:

ID	Deskripsi	Prioritas	Catatan
R5	Tambahkan error code yang hilang (E1001, E2001, E3002)	Menengah	Perlu mapping error code lintas tahap pipeline
M-01	Tambahkan source location di semua AST nodes	Menengah	Perlu modifikasi ast-factory.js dan semua parser
M-03	Tambahkan unit test untuk compiler statements	Menengah	Perlu test fixture baru untuk setiap statement
M-05	Dokumentasi API dan arsitektur pipeline	Rendah	Dokumentasi teknis untuk kontributor

11. File yang Diubah

Berikut adalah daftar file yang dimodifikasi dan lokasi perubahannya:

File	Perubahan	Item
analyzer/karsa-analyzer.js	checkWriteToTurunan, lookupSymbol, checkTypeHint, visitData/Tetap/UbahDeclaration	R3, R4
parser/expression-parser.js	Module-level cache getStmtParser() menggantikan inline require	H-01
engine/karsa.js	Mode recover dengan options.recover, multi-stage error collection	H-02
compiler/karsa-compiler.js	Runtime helpers: __effectMap, __cleanup, __deps tracking, safe iteration	M-02
utils/visitor.js	getChildKeys: AttributeNode, Parameter, SelfReference; nodeTypes: +SelfReference	M-04